Stokvel Database Schema (PostgreSQL)

1. users

```
CREATE TABLE users (
  user_id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  surname VARCHAR(100) NOT NULL,
  date_created TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- **Purpose:** Stores personal details of each member.
- **Constraints:** user_id is the primary key.

---

2. user_account

```
CREATE TABLE user_account (
  account_id SERIAL PRIMARY KEY,
  user_id INT UNIQUE NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  account_num VARCHAR(50) UNIQUE NOT NULL,
  amount NUMERIC(12,2) DEFAULT 0
);
```

- **Purpose:** Holds financial account details for each user.
- **Constraints:**
  - user_id is **UNIQUE** → enforces one-to-one relationship (each user has exactly one account).
  - account_num is **UNIQUE** → prevents duplicate account numbers.
  - ON DELETE CASCADE → deleting a user also deletes their account.

---

3. user_transact

```
CREATE TABLE user_transact (
  transact_id SERIAL PRIMARY KEY,
  account_id INT NOT NULL REFERENCES user_account(account_id),
  amount_transacted NUMERIC(12,2) NOT NULL,
  transaction_type VARCHAR(20) NOT NULL
    CHECK (transaction_type IN ('deposit','withdraw','transfer')),
  datetime TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  goal_id INT REFERENCES goal(goal_id)
);
```

- **Purpose:** Records all deposits, withdrawals, and transfers.
- **Constraints:**
  - account_id FK → links transactions to accounts.
  - transaction_type restricted to three values.
  - goal_id FK → links deposits to a savings goal.

---

4. goal

```
CREATE TABLE goal (
  goal_id SERIAL PRIMARY KEY,
  goal_name VARCHAR(255) NOT NULL,
  goal_amount NUMERIC(12,2) NOT NULL,
  goal_term VARCHAR(100),
  total_transact NUMERIC(12,2) DEFAULT 0,
  remaining_amount NUMERIC(12,2) DEFAULT 0,
  status VARCHAR(20) NOT NULL DEFAULT 'active'
    CHECK (status IN ('active','matured','closed'))
```

);

- **Purpose:** Defines group savings targets.
- **Constraints:**
    - status tracks lifecycle: active → matured → closed.
    - total_transact and remaining_amount are updated automatically via triggers.

---

5. Triggers

Prevent withdrawals until goal matured

```
CREATE OR REPLACE FUNCTION prevent_withdrawals()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.transaction_type = 'withdraw' AND NEW.goal_id IS NOT NULL THEN
    IF (SELECT status FROM goal WHERE goal_id = NEW.goal_id) <> 'matured' THEN
      RAISE EXCEPTION 'Withdrawals not allowed until goal is matured.';
    END IF;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_withdrawals
BEFORE INSERT ON user_transact
FOR EACH ROW
EXECUTE FUNCTION prevent_withdrawals();
```

Auto-update goal totals on deposits

```
CREATE OR REPLACE FUNCTION update_goal_totals()
```

```
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.transaction_type = 'deposit' AND NEW.goal_id IS NOT NULL THEN
    UPDATE goal
    SET total_transact = total_transact + NEW.amount_transacted,
      remaining_amount = goal_amount - (total_transact + NEW.amount_transacted)
    WHERE goal_id = NEW.goal_id;

    UPDATE goal
    SET status = 'matured'
    WHERE goal_id = NEW.goal_id AND total_transact >= goal_amount;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_goal_totals
AFTER INSERT ON user_transact
FOR EACH ROW
EXECUTE FUNCTION update_goal_totals();
```

Relationships Summary

- **users → user_account**: 1:1 (enforced by UNIQUE user_id).
- **user_account → user_transact**: 1:N (one account can have many transactions).
- **goal → user_transact**: 1:N (one goal can have many deposits).
- **Withdrawals**: blocked until goal status = matured.

- **Deposits**: automatically update total_transact and remaining_amount.
